Тема 13: ООП. Основни принципи. Класове и обекти. Наследяване и капсулация.

ТЕОРИЯ

1. Основни принципи на ООП

Принцип	Описание
Абстракция	Разделяне на представянето от използването. Потребителят работи с обекта чрез интерфейс, без да знае вътрешното представяне.
Капсулация	Скриване на вътрешните данни; достъп само чрез контролиран интерфейс (getters/setters).
Наследяване	Клас наследява свойства и поведение от друг клас.
Полиморфизъм	Различни класове могат да се ползват по един и същ начин (виж Тема 14).

---

2. Класове и обекти

Клас = съвкупност от данни (член-полета) и функции (методи), обвързани в обща логическа структура.
Обект = конкретна инстанция на класа.

```
class Point {
public:
    int x;
    int y;

    void print() const {           // const метод — не променя полетата
        cout << x << " " << y << endl;
    }
};

Point p1;           // обект — декларация
Point p2 = {3, 4};  // инициализация (aggregate)
p1.x = 5;
p1.print();
```

`this` — указател към текущия обект в тялото на метода.

---

3. Модификатори за достъп

Модификатор	**Достъп**
`private`	Само в самия клас
`protected`	В класа и в наследниците
`public`	Отвсякъде

> По подразбиране в `class` всичко е `private`. В `struct` — `public`.

---

4. Конструктори

Конструкторите имат **същото име** като класа, **без тип на връщане**.

Конструктор по подразбиране (default)

```
class A {
public:
    A() { /* инициализация */ }
};
A obj;         // извиква default конструктора
A arr[10];     // 10 пъти default конструктор
```

> Ако е деклариран **друг** конструктор, компилаторът НЕ генерира default автоматично.

Конструктор с параметри

```
class Point {
    int x, y;
public:
    Point(int x, int y) : x(x), y(y) {}  // member initializer list
};
Point p(3, 4);
```

Конструктор за копиране (copy constructor)

```
A(const A& other) { /* копира other */ }

A obj1;
A obj2(obj1);     // copy constructor
A obj3 = obj1;    // copy constructor (НЕ operator=!)
void f(A obj) {}
f(obj1);          // copy constructor (при предаване по стойност)
```

> Компилаторът генерира default — **memberwise copy** (поле по поле).

Move конструктор

```
A(A&& other) { /* прехвърля ресурси */ }

A obj2 = std::move(obj1);  // извиква move конструктора
```

Прехвърля ресурси вместо да ги копира — по-ефективен.

---

5. Деструктор

```
~ClassName() { /* почиства ресурси */ }
```

- Извиква се **автоматично** при унищожаване на обекта (край на scope, `delete`)
- Един — без параметри, без тип на връщане, не може да се претоварва
- Компилаторът генерира тривиален деструктор, ако не е деклариран

---

6. Правилото на трите (Rule of Three)

Ако класът управлява ресурс (динамична памет, файл, мрежова връзка), трябва да се дефинират **и трите**:

1. **Деструктор** — освобождава ресурса
2. **Конструктор за копиране** — прави дълбоко копие
3. **Оператор за присвояване** — прави дълбоко копие + самоприсвояване

Rule of Five (C++11): добавят се move constructor и move assignment operator.

```
class MyString {
    char* str;

    void copy(const char* other) {
        str = new char[strlen(other) + 1];
        strcpy(str, other);
    }

public:
    // Конструктор по подразбиране
    MyString() {
        str = new char[1];
        str[0] = '\0';
    }

    // Конструктор с параметри
    MyString(const char* s) { copy(s); }

    // Конструктор за копиране
    MyString(const MyString& other) { copy(other.str); }

    // Оператор за присвояване (Rule of 3)
    MyString& operator=(const MyString& other) {
        if (this != &other) {    // предпазване от self-assignment!
            delete[] str;
            copy(other.str);
        }
        return *this;
    }

    // Деструктор
    ~MyString() { delete[] str; }

    const char* get() const { return str; }
};
```

---

7. RAII (Resource Acquisition Is Initialization)

Принцип: Жизненият цикъл на ресурса е обвързан с жизнения цикъл на обекта.

- **Конструктор**: заделя ресурса
- **Деструктор**: освобождава ресурса

Предимства: ресурсът винаги се освобождава, дори при изключения.

Примери НЕ-RAII (трябва ръчно управление): `FILE*`, `lock()/unlock()`, `open()/close()`.

---

8. Методи — декларация и предаване на параметри

```
class Test {
    int value;
public:
    // const метод – не може да мени член-данни
    int getValue() const { return value; }

    // мутатор
    void setValue(int v) { value = v; }

    // по референция – ефективно, без копие
    void process(const Test& other) { /* ... */ }
};
```

Член-функциите имат имплицитен параметър **`this`** — указател към текущия обект.

---

9. Наследяване

```
class Base {
public:    int x;
protected: int y;
private:   int z;  // НЕДОСТЪПНО в наследника
};

class Derived : public Base {
    // x → public
    // y → protected
    // z → недостъпно
};
```

Три вида наследяване:

Тип	**public члена на Base**	**protected члена на Base**
`: public`	public	protected
`: protected`	protected	protected
`: private`	private	private

По подразбиране в `class` → `private` наследяване. В `struct` → `public`.

Ред на извикване при създаване на обект:

1. Конструктор на Base
2. Конструктор на Derived

Ред при унищожаване:

1. Деструктор на Derived
2. Деструктор на Base

```
class Animal {
public:
    Animal(const string& name) : name(name) {}
    string name;
};

class Dog : public Animal {
public:
    Dog(const string& name, const string& breed)
        : Animal(name),  // извикваме конструктора на Base
          breed(breed) {}
    string breed;
};
```

---

10. Конструктор за копиране при наследяване

```
// При копиране на наследен клас — трябва ЯВНО да извикаме copy constructor на
Base:
Derived(const Derived& other) : Base(other) {
    // ... копираме само Derived-специфичните полета
}

// Оператор за присвояване:
Derived& operator=(const Derived& other) {
    if (this != &other) {
        Base::operator=(other);  // явно извикваме Base::operator=
        // ... присвояваме Derived полетата
    }
    return *this;
}
```

---

11. Капсулация — Getters и Setters

```
class BankAccount {
    double balance;  // private — директен достъп забранен

public:
    // Selector (getter) — само четене
    double getBalance() const { return balance; }

    // Mutator (setter) — с валидация
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
```

```
};
```

---

12. Статични полета и методи

```
class Counter {
public:
    static int count;  // споделено между всички обекти!
    int id;

    Counter() { id = ++count; }

    static int getCount() {  // без this!
        return count;
        // НЕ може да достъпва нестатични полета
    }
};
int Counter::count = 0;  // задължително дефиниране извън класа

Counter c1, c2;
cout << Counter::count;   // 2
cout << Counter::getCount(); // 2
```

---

13. Вложени класове и Композиция

Вложен клас (nested): дефиниран в scope-а на друг клас.

```
class Triangle {
public:
    class Point {       // Triangle::Point
        int x, y;
    };
    Point vertices[3];
};
Triangle::Point p;
```

Композиция: обект е член-поле на друг клас.

```
class Engine { /* ... */ };
class Car {
    Engine engine;   // Car "има" Engine
};
```

КАК СЕ РЕШАВАТ ЗАДАЧИ

Тема 13 е директно свързана с **Task 2** на практическия изпит (ООР C++).

Тип 1: Проследяване на конструктори/деструктори

Стратегия: следи кога се извикват конструкторите и деструкторите.

```
class A {
public:
```

```
    A()             { cout << "A()" << endl; }
    A(const A&)     { cout << "A(copy)" << endl; }
    ~A()            { cout << "~A()" << endl; }
};

class B : public A {
public:
    B()  { cout << "B()" << endl; }
    ~B() { cout << "~B()" << endl; }
};

int main() {
    B b;        // A() → B()
}               // ~B() → ~A()

void f(A a) {} // при извикване: A(copy); при край: ~A()
```

Правила за ред:

- **Създаване**: Base → Derived (отгоре надолу в йерархията)
- **Унищожаване**: Derived → Base (обратното)
- **Член-обекти**: се конструират по ред на декларация, деструктират в обратен ред

Тип 2: Имплементация на клас с Rule of Three

Стандартна задача: Напиши клас с динамична памет.

```
class IntArray {
    int* data;
    int  size;

public:
    // Default
    IntArray() : data(nullptr), size(0) {}

    // Конструктор с параметри
    IntArray(int n) : size(n) {
        data = new int[n]();  // () = инициализира с 0
    }

    // Copy constructor
    IntArray(const IntArray& other) : size(other.size) {
        data = new int[size];
        for (int i = 0; i < size; i++) data[i] = other.data[i];
    }

    // Operator=
    IntArray& operator=(const IntArray& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new int[size];
            for (int i = 0; i < size; i++) data[i] = other.data[i];
        }
        return *this;
    }

    // Destructor
    ~IntArray() { delete[] data; }
```

```
    int& operator[](int i) { return data[i]; }
    int  getSize() const   { return size; }
};
```

Тип 3: Наследяване — достъп и покриване

```
class Shape {
protected:
    double area;
public:
    Shape(double a) : area(a) {}
    double getArea() const { return area; }
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : Shape(3.14159 * r * r), radius(r) {}
    double getRadius() const { return radius; }
};

Circle c(5.0);
cout << c.getArea();    // достъпно — public наследяване
cout << c.radius;       // ГРЕШКА — radius е private
```

Бърза справка

Концепция	**Ключов факт**
`class` vs `struct`	Разлика: default `private` vs `public`
Default конструктор	Генерира се ако няма НИКАКЪВ друг конструктор
Copy constructor	`A(const A&)` — извиква се при копиране
`A obj2 = obj1`	Copy constructor (не operator=!)
Rule of Three	Деструктор + copy ctor + operator=
RAII	Ресурс в ctor, освобождаване в dtor
Self-assignment	`if (this != &other)` в operator=
`return *this`	В operator= за chaining
Ред ctor	Base → Derived → member fields
Ред dtor	member fields → Derived → Base
`static` поле	Споделено, дефинира се извън класа
`static` метод	Без `this`, само статични членове
`const` метод	Не мени полетата
Деструктор при наследяване	Derived dtor автоматично вика Base dtor